

Communication-Matched Warp Collectives for ML-KEM and ML-DSA on a RISC-V SIMT GPU

Anonymous Authors

Abstract—ML-KEM and ML-DSA share Keccak and number-theoretic transforms (NTTs), yet their useful parallelism has incompatible communication: Keccak repeatedly reduces and permutes a 25-word state, while an NTT exchanges coefficient pairs at power-of-two distances. We present communication-matched warp collectives that keep both states in ordinary lane registers of an RV32 RISC-V GPU. The instructions are stateless, use standard source vectors and single-destination writeback, and implement fixed Keccak neighborhoods or paired NTT butterflies. A serialized modular unit shares its routing and multiplier bank across the 16-bit ML-KEM and 32-bit ML-DSA datapaths. On a 200-MHz V80, Stage plus shared NTT speeds complete ML-KEM-768 by $2.357\times$ at one request and $2.044\times$ at eight relative to matched cooperative software. Full-RAM ML-DSA-65 reaches $1.984\times$ and $1.910\times$ with the same units and mapped pointwise work. A matched five-point sweep selects two multipliers: it removes 87.500% of NTT-hierarchy DSPs at a 2.963% standalone-cycle and at most 0.143% complete-request penalty. The results show that GPU cryptographic hardware must be sized at request scope, not by primitive peak throughput.

Index Terms—post-quantum cryptography, ML-KEM, ML-DSA, Keccak, NTT, SIMT, RISC-V

I. INTRODUCTION

ML-KEM and ML-DSA standardize lattice-based key establishment and digital signatures [1], [2]. Their deployments must process independent requests while retaining the exact rejection sampling, encoding, and control of the standards. A programmable SIMT processor is attractive because a warp can cooperate within one primitive and different warps can execute independent requests. The challenge is data movement inside the warp.

The two shared primitives expose different graphs. An NTT level contains 128 independent butterflies, but its partner distance changes at every level. Keccak-f[1600] instead applies column reductions, a fixed word permutation, and row-neighbor logic to 25 tightly coupled 64-bit words. Generic lane shuffles can express both graphs, but software pays an instruction and dependence cost for every edge. A pointer accelerator removes more instructions by moving a complete state to private storage, at the cost of transfers, memory-system integration, and a new scheduling boundary.

Prior optimized RISC-V software and scalar extensions accelerate modular butterflies and Keccak rounds [3]–[5]; GPU software distributes Keccak over 25 threads [6]; and a patent describes a warp-level Keccak round [7]. These results do not answer three questions for an open SIMT core: how a collective with multiple logical outputs fits single-destination writeback, how ML-KEM and ML-DSA share one NTT

datapath, and whether primitive throughput predicts complete-request performance.

We answer these questions in Vortex [8] with a common rule: place a primitive in lane registers, then implement only the fixed edges of its communication graph (Fig. 1). Our contributions are:

- A stateless warp-collective interface for Keccak stages and XOR-paired NTT butterflies. Each lane receives one result through normal dependency, backpressure, and writeback machinery.
- A shared K/D NTT unit whose pair routing and configurable multiplier bank support seven-layer 16-bit ML-KEM and eight-layer 32-bit ML-DSA. A five-point RTL and post-route sweep selects two multipliers.
- A controlled evaluation from instruction use to complete requests. It separates mapping, stage and round fusion, pointer offload, arithmetic sharing, and physical cost instead of attributing all gains to a new instruction.

II. BACKGROUND AND MOTIVATION

A. Two workloads, two communication graphs

Both standards operate on degree-256 polynomials, but ML-KEM uses seven NTT layers modulo $q_K = 3329$ with 16-bit coefficients, whereas ML-DSA uses eight layers modulo $q_D = 8380417$ with 32-bit coefficients. With lane l holding $a[l + 32k]$, $0 \leq k < 8$, the first three layers exchange registers within one lane. The remaining layers exchange lane l with $l \text{ xor } d$; $d = 16, 8, 4, 2$ for ML-KEM and additionally $d = 1$ for ML-DSA. This layout gives coalesced loads and stores and keeps every intermediate coefficient in GPRs.

Both schemes also call SHA-3/SHAKE [9]. Mapping Keccak word $A[x, y]$ to lane $x + 5y$ exposes the 25-word state, but a round still needs three distinct patterns: five-lane column parity (θ), a rotate-and-gather permutation ($\rho\pi$), and three-lane row logic ($\chi\iota$). On RV32 every 64-bit word occupies two GPRs and rotations cross their boundary. A useful instruction must therefore see both halves while returning only one architectural word.

B. Why the execution boundary matters

Register residence removes inter-stage memory traffic, but it does not by itself remove communication instructions. The fair baseline is consequently a cooperative shuffle kernel with the same state placement, rather than a single-lane scalar routine. Conversely, fusing more work is not always better: it shortens the instruction stream but enlarges the unit and can serialize independent warps. A pointer engine makes the opposite trade,

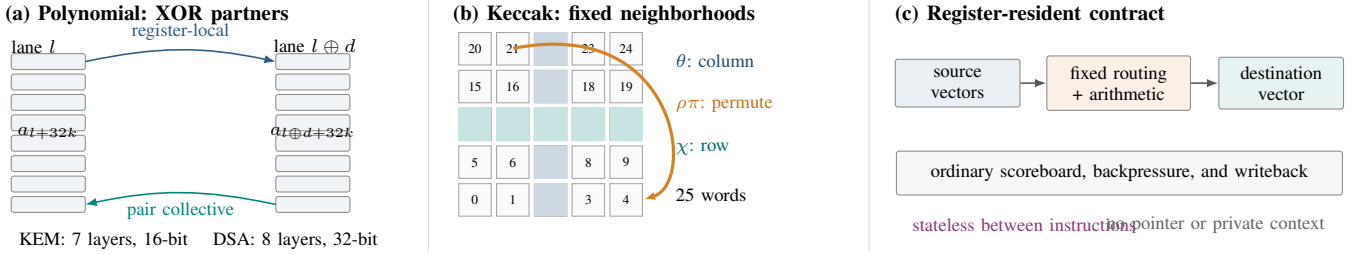


Fig. 1. The communication determines the hardware boundary. NTT coefficients use register-local or XOR-paired partners; Keccak uses fixed column, permutation, and row neighborhoods. Both execute through a stateless register contract, so the processor retains ordinary scheduling and writeback.

removing a whole permutation from the ALU path while exposing state transfers through the LSU. We evaluate all three boundaries under identical request semantics.

This reasoning also applies to the multiplier bank. A 32-lane paired butterfly needs 16 products, but peak issue rate is useful only while NTT work is on the request critical path. NTT-PIM and BP-NTT similarly choose their operation boundary from physical data placement [10], [11]. We apply that principle to the warp register file and size the arithmetic with end-to-end measurements. More generally, PacQ and SynGPU show the value of matching SIMT hardware to irregular dataflow [12], [13]; our focus is fixed cryptographic communication with exact software compatibility.

C. Relation to prior cryptographic accelerators

Scalar PQC extensions optimize a different register boundary. RISQ-V keeps polynomial state in wide scalar registers and fuses several Kyber layers, whereas Nannipieri et al. use twiddle state and packed butterfly outputs to fit a scalar instruction stream [4], [5]. Those encodings avoid a SIMT lane exchange but cannot exploit the 128 independent butterflies already present in every transform level. Our pair instruction instead treats two lanes as the natural producer of two results while preserving one destination per lane.

GPU mappings expose more parallelism but usually leave communication in software. ML-Cube maps Keccak to 25 threads and uses shuffles around machine-learning accelerators [6]; the NVIDIA collective-round description places a full 64-bit round behind a warp instruction [7]. We retain the 25-lane layout but compare three implementation boundaries: software shuffles, stage collectives, and a fused round. This comparison is necessary on RV32, where low/high results double the architectural operations and a fused unit duplicates internal work to remain stateless.

Dedicated accelerators can hide these interface costs with private SRAM and custom schedules. CHAM, NTT-PIM, BP-NTT, and ZK-Flex demonstrate the performance of co-designed polynomial storage and compute [10], [11], [14], [15]. Their boundary is appropriate when the accelerator owns the application state. Our target retains sampling, packing, rejection control, and both standardized algorithms on one programmable core; the added units therefore operate on the same GPR state as the surrounding code. The resulting

question is not whether a custom primitive is faster, but which fixed communication is worth adding once software mapping, concurrency, and complete-request coverage are held constant.

D. SIMT constraints and design requirements

Vortex issues one instruction for a warp and carries a thread mask with the operand vectors. Each lane reads the same architectural register names, while the result arbiter and scoreboard treat the returned lane vector as one destination. A collective may use values from several lanes internally, but it must still retire with those ordinary semantics. Retaining a partial round or transform inside the unit would instead require a new context to be saved on a warp switch; writing two architectural destinations would require a new scoreboard and recovery rule. We therefore require one explicit transaction to consume source vectors, compute a fixed communication step, and return one value per active lane.

The active mask must contain every producer needed by that step. For Keccak, the 25 state lanes execute as one converged group and the unused lanes are padding. For an NTT at distance d , activity must be closed under $l \mapsto l \text{ xor } d$; otherwise one lane of a butterfly would read an absent partner. Distance, direction, modulus, and Keccak round index are therefore explicit instruction fields rather than hidden unit state. These rules are simple enough to assert in simulation and allow a stalled collective to carry the same warp ID, mask, and destination metadata as the existing variable-latency units.

The evaluation follows the same contract. We compare scalar code, register-resident shuffles, stage collectives, a fused round, and pointer offload while preserving the library API and request input. For NTT, changing multiplier count changes only internal scheduling; the opcode, layout, and binary remain fixed. A useful choice must improve complete requests for both schemes, close full-core timing, and preserve model parity.

III. DESIGN AND IMPLEMENTATION

A. Collective contract and core integration

Figure 2 places the units beside the integer ALU. An instruction consumes complete source vectors and the active mask, then returns one destination vector through the existing result arbiter. No operation contains a pointer, writes memory, or retains architectural state after writeback. Keccak requires

a converged warp; NTT requires pair-closed masks. Padding lanes return zero. These constraints permit static wiring and allow different warps to interleave without saving a cryptographic context.

The decoder uses one custom opcode family and a compact function field for operation, low/high selection, NTT direction, modulus, and XOR distance. The scoreboard sees the same source and destination registers as the execution unit; no implicit register or memory dependence is introduced. A collective accepts an instruction only when its entire input vector and metadata can be captured. It holds warp ID, destination, mask, and lane results under output backpressure, so a stalled result cannot be paired with a later warp. This property is especially important for a serialized NTT instruction, which occupies several internal beats but commits exactly once.

B. Keccak: stages, rounds, and RV32 halves

For coordinates modulo five, the datapath implements

$$\begin{aligned} C[x] &= \bigoplus_y A[x, y], \\ T[x, y] &= A[x, y] \oplus C[x-1] \oplus \text{rol}(C[x+1], 1), \\ B[y, 2x+3y] &= \text{rol}(T[x, y], \rho[x, y]), \\ A'[x, y] &= B[x, y] \oplus (\neg B[x+1, y] \wedge B[x+2, y]). \end{aligned} \quad (1)$$

The *Stage* interface exposes θ , $\rho\pi$, and $\chi\iota$. Each has low/high RV32 forms; the two forms read the same old word pair and return the selected half. Six issues form a round. The *Round* interface fuses all stages and uses a literal round index. Its low/high instructions recompute the internal full word from identical operands, avoiding hidden state and multi-destination writeback. Two issues form an RV32 round. A memory-coupled engine that reads and writes the 200-byte state provides the coarser comparison point.

Stage has two pipeline registers and accepts one operation per cycle. Round registers parity, θ , $\rho\pi$, and $\chi\iota$ in four steps and also has initiation interval one. Its larger internal state explains why it can improve an isolated permutation while producing little request-level gain. For θ and $\rho\pi$, both half instructions read both old halves because rotation carries cross the 32-bit boundary. Software therefore writes the first result to a temporary until the second issue has consumed the old pair. $\chi\iota$ reads one half and a warp-uniform round index. Round encodes the index as an immediate, enabling the 24 rounds to be expanded without a table load or branch in the permutation body.

C. NTT: one product, two lane results

For an XOR pair, pair-low lane p supplies a and twiddle ζ ; pair-high lane $h = p \text{ xor } d$ supplies b . A forward instruction computes

$$t = \text{Mont}(b\zeta), \quad (r_p, r_h) = (a + t, a - t). \quad (2)$$

One product thus serves both lanes, and each lane writes its own destination. The inverse instruction first forms a sum

TABLE I
COLLECTIVE INTERFACES ON RV32. ISSUE COUNTS ARE PER KECCAK ROUND OR PER CROSS-LANE NTT LAYER; B IS THE SERIALIZED BEAT COUNT.

Operation	Cooperation	Result/lane	Issues or II
$\theta, \rho\pi, \chi\iota$	25 lanes	word half	6/round
Round	25 lanes	word half	2/round
NTTB.F.K/D	XOR pair	coefficient	$\Pi=B$
NTTMUL.K/D	lane-local	product	$\Pi=B$

and difference, then multiplies the appropriate branch. Lane-local NTTMUL.K/D handles the first three layers, inverse scaling, ML-KEM base multiplication, and independent ML-DSA pointwise products.

A bank of M signed 32×32 multipliers serializes the 16 logical pair products without changing the ISA or binary. CT and ML-DSA inverse butterflies use $16/M$ beats; ML-KEM inverse and 32-lane scalar products use $32/M$. The direct latency and initiation interval are $6 + B$ and B , where B is the beat count. Operands are captured on the first handshake, lane results are assembled by index, and backpressure freezes the serializer and product pipeline. Pair routing, capture, and writeback are shared; only the constant-arithmetic reductions for q_K and q_D differ.

The shared bank preserves each library's native residue representation rather than converting coefficients at its boundary. For ML-KEM, operands are sign-extended from 16 bits; a shift-add Montgomery factor and the constant 20159 Barrett product produce the forward and inverse results. ML-DSA uses the full signed 32×32 product, its Montgomery inverse, and q_D in the reduction stage. Both modes therefore use the same operand capture, multipliers, beat controller, and result assembly. Modulus selection changes only the constant paths after multiplication. This organization lets NTTMUL.D serve both butterfly products and ordinary pointwise products without a second arithmetic unit.

Twiddles remain explicit lane operands. The unit contains neither a twiddle ROM nor a layer counter, so it does not assume a transform schedule or retain state between issues. Software can use the same operation for forward and inverse transforms, final scaling, ML-KEM base multiplication, and ML-DSA pointwise work. More importantly, the scheduler may suspend a warp between layers and issue another warp without saving an accelerator context.

Figure 3 makes the two encoding constraints explicit. A logical butterfly has three values (a, b, ζ) and two results, but the pair-low lane provides the twiddle and the two physical lanes independently receive r_p and r_h . This converts an awkward three-input/two-output scalar operation into an ordinary two-vector/one-vector SIMT transaction. For M2, the routing network presents two logical products per beat for eight beats. Result assembly records their lane indices and releases the vector only after the final reduction; later warps can still execute other functional units while the issuing warp waits on the destination dependency.

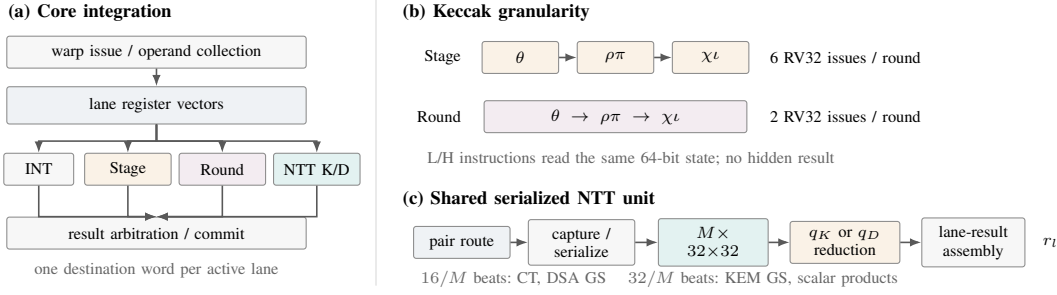


Fig. 2. Microarchitecture. The collectives are ordinary ALU processing elements. Keccak exposes either three two-stage operations or a fused four-stage round. The K/D NTT unit captures a lane vector, serializes products over M multipliers, selects a modulus-specific reduction, and assembles one result per lane.

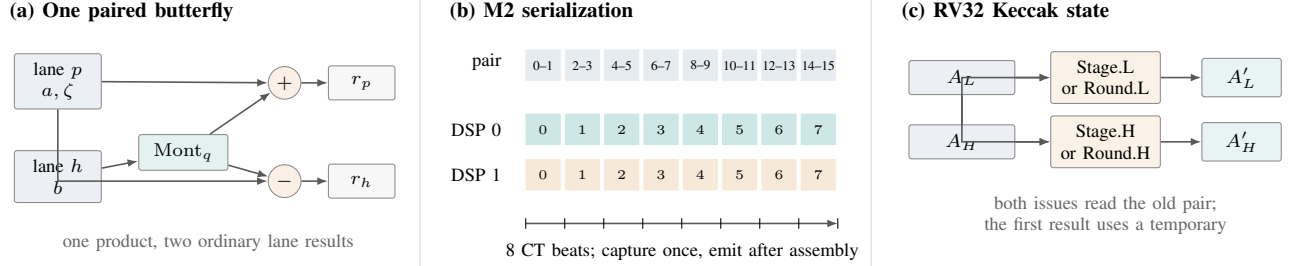


Fig. 3. Datapath details. (a) A lane pair turns one Montgomery product into two ordinary results. (b) M2 covers 16 logical products in eight beats without changing the ISA. (c) RV32 Keccak low/high instructions both read the old 64-bit state; a temporary preserves it across the two issues.

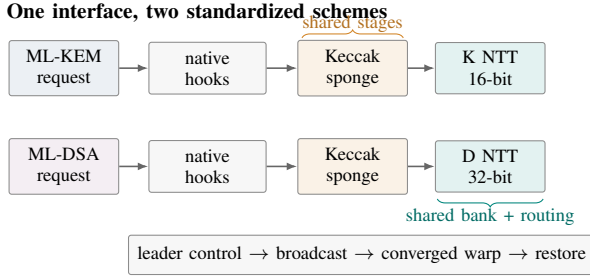


Fig. 4. Software integration. Both native libraries retain leader control and enter converged warp kernels through their existing Keccak and NTT hooks. The algorithms select a reduction mode while sharing routing and multipliers.

D. Complete-library integration

The native libraries retain their public APIs and byte-level outputs [16], [17]. Lane zero executes scalar control, broadcasts primitive arguments, activates the warp, and restores leader execution (Fig. 4). One-shot SHAKE keeps its state in registers across absorb, permutation, and squeeze; an incremental context is stored between calls. NTT kernels load eight coefficients per lane and perform all layers without intermediate memory. ML-DSA also reuses `NTTMUL.D` for ordinary pointwise products; its five-term matrix-vector sum accumulates in 64-bit software and performs one final Montgomery reduction, preserving the library representation.

Cooperation is confined to primitive kernels. All 32 lanes enter and leave a region together; Keccak activates lanes 0–24, while an NTT butterfly uses a pair-closed mask at its encoded

XOR distance. The leader performs allocation, sampling decisions, and byte-oriented packing outside those regions. This keeps rejection loops and public interfaces unchanged and prevents a collective from observing a partially active pair. The compiler intrinsics name every source and destination, so the existing scoreboard orders a later consumer exactly as it does for an integer ALU result.

One-shot sponges and transforms keep state in GPRs between initial load and final store. Pointer Keccak instead transfers the 200-byte state through the LSU, allowing independent ALU work from another warp.

The full-RAM ML-DSA-65 configuration owns an 88-KiB request arena. Leader-only allocation and reverse-order release reclaim temporaries after rejected signatures; eight distinct inputs exercise 2–11 signing attempts. Simulation assertions check converged Keccak masks, pair-closed NTT masks, uniform round indices, backpressure, and warp metadata. Unit tests cover all pair distances and FIPS-202 rounds. Complete ML-KEM-768 and ML-DSA-65 runs compare keys, ciphertexts or signatures, and shared secrets byte for byte with portable C.

E. Model and RTL verification

The SimX timing model mirrors every RTL bank beat, pipeline latency, and result stall. This provides a functional oracle before processor RTL runs and makes cycle disagreement observable rather than hidden by a looser simulator. The processor tests then exercise instruction decode, scoreboard hazards, masks, and writeback arbitration. XRT adds the AFU

TABLE II
PRIMARY EXPERIMENTAL CONFIGURATION.

Processor	RV32IM, one core, F/D disabled
Warp organization	8 resident warps, 32 lanes
On-core memory	16 KiB I\$, D\$, and local memory
Workloads	ML-KEM-768; full-RAM ML-DSA-65
Timing	XRT RTL integration; SimX controls; V80 silicon
Physical flow	Alveo V80, 200 MHz, Vivado 2025.1

integration used by the complete applications. We require identical retired instruction counts for matched SimX/XRT binaries and keep timing gaps below the pre-defined 5% parity bound; we do not widen the bound to accommodate a new unit.

Unit testbenches cover all five XOR distances, both directions and moduli, masks, backpressure, and consecutive warp metadata. Instruction tests compare every lane with scalar references; transform boundary vectors precede complete library KATs. Independent post-route cores measure each hardware choice.

IV. EVALUATION

A. Methodology

All measurements use one RV32IM Vortex core with eight resident 32-thread warps, 16-KiB instruction/data caches and local memory, no L2/L3, and F/D disabled. XRT exercises the processor RTL and accelerator interface; SimX is used for controlled ablations. Matched binaries retire identical instruction counts, and the reported parity cases remain within their 5% cycle bound. Independent core implementations and the complete AVED image use Vivado 2025.1 OPT3 and a 200-MHz target. The board clock is read back at 200 MHz; one request measures latency and eight independent requests measure batch makespan, with fixed eight-warp batches extending to 64 requests. A complete request includes keypair, encapsulation, and decapsulation for KEM or keypair, signing, and verification for DSA.

B. Communication collectives improve complete requests

Figure 5 reports five-run medians from one resident board image. Holding KEM’s arithmetic mapping fixed, Stage gives 2.078/1.850 $\times$ at one/eight requests; shared NTT then adds 1.134/1.105 $\times$, for 2.357/2.044 $\times$ overall. For full-RAM DSA, Stage gives 1.624/1.539 $\times$, shared NTT adds 1.115/1.197 $\times$, and mapped pointwise work adds 1.095/1.037 $\times$, reaching 1.984/1.910 $\times$ overall. Every board arm passes byte-exact known-answer checks. These ratios use matched board controls, and all arms select software paths within the same hardware image.

Controlled XRT runs isolate the Keccak boundaries. A cooperative Shuffle arm reduces one-request KEM cycles from 24.67 million for optimized PQRV to 7.91 million. Relative to Shuffle, Stage reaches 1.462 $\times$ at one request and 1.552 $\times$ at eight; Round reaches 1.520 $\times$ /1.583 $\times$. Round is 1.130 $\times$ faster than Pointer at one request, whereas Pointer is 1.374 $\times$ faster at eight. The counters do not separate scheduler from cache effects, so we do not assign this crossover a single cause.

With Pointer fixed in a separate XRT cohort, shared K/D NTT instructions save 4.913% of the complete KEM round trip and 5.939% of the DSA round trip over matched W32 register-shuffle NTTs. Reusing `NTTMUL.D` and distributing the five-term sum reduces DSA cycles by 12.538% at one request and 11.687% at eight, with no additional RTL. These gains use separate matched denominators and are not additive.

A second 200-MHz V80 sweep covers ML-KEM at 512, 768, and 1024 and ML-DSA at 44, 65, and 87. Across these six parameter sets, Stage, shared NTT, and mapped arithmetic gain 1.454–1.552 $\times$ at one request and 1.677–1.832 $\times$ at eight over matched cooperative software (Fig. 7). This cohort fixes a different software mapping from Fig. 5, so its ratios are not combined with that figure. All 120 parameter/concurrency cells pass byte-exact KATs on the same image.

In the measured request, ML-KEM invokes 15 forward and nine inverse transforms; ML-DSA invokes 46 and 49. The shared unit thus exercises different layer counts, twiddle schedules, inverse conventions, and reduction paths, including KEM’s second product and DSA’s signed 32-bit arithmetic.

C. Primitive peaks do not select the hardware

Figure 6(a) compares Round with equally unrolled Stage. At one and eight active warps, fusion improves dependent permutation throughput by 3.168 $\times$ and 3.291 $\times$, respectively. Its matched complete-KEM gains are only 1.028 $\times$ and 1.033 $\times$. After Stage acceleration, measured permutation time is only 4.964% of an instrumented request; absorb, squeeze, sampling, packing, and control bound the remaining benefit.

M16 is the fastest NTT bank, but M8 and M4 increase standalone NTTBF cycles by only 0.181% and 0.715%. M2 increases them by 2.963% while removing 71.074% of NTT LUTs, 46.131% of flip-flops, and 87.500% of DSPs. Across ML-KEM/ML-DSA at one/eight requests, its largest XRT makespan increase is 0.143%. M1 saves only five additional core DSPs but raises standalone cycles by 25.043% and increases NTT flip-flops due to deeper serialization. We therefore select M2, despite M16 having the highest primitive throughput.

All five standalone runs retire 59,534 instructions and pass 1,044 butterfly vectors; only internal bank timing changes. SimX and RTL differ by at most 0.959%, so the flat complete-request curve is not a functional or instruction-count artifact. It reflects available latency hiding and the small fraction of request time that remains in the shared NTT unit. M1 exposes the limit: once serialization grows to 16 or 32 beats, primitive cycles rise sharply while end-to-end area savings have nearly saturated.

Table III closes every core without routing errors. At M16, Stage adds 1.418% LUTs and 1.504% FFs; Round adds 1.226% LUTs and 4.463% FFs; Pointer adds 4.124% LUTs and 2.166% FFs. With Keccak fixed at the software base, M2 reduces whole-core LUTs, FFs, and DSPs by 4.190%, 3.771%, and 39.773% relative to M16. The routed M2+Stage core directly measures their joint cost; Stage adds 1.289% LUTs and 1.570% FFs to M2 alone. The separate routed board

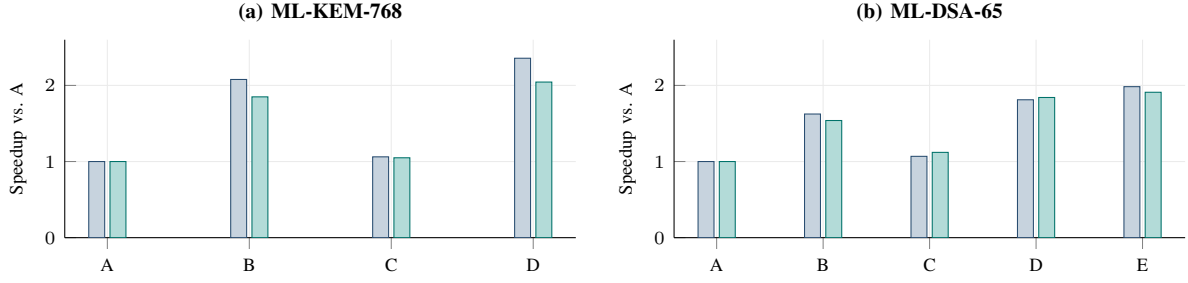


Fig. 5. 200-MHz V80 complete-request speedups over matched cooperative software (A). B enables Keccak Stage; C enables shared NTT; D enables both; E also maps DSA pointwise arithmetic. Blue and teal bars show one and eight requests, respectively. Speedups divide five-run median makespans.

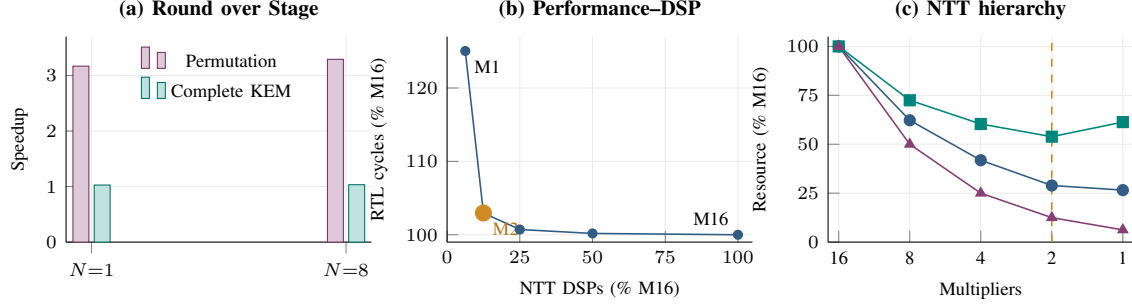


Fig. 6. Selecting granularity and multiplier count. (a) Round fusion's primitive and request gains. (b,c) Five banks run the same NTTBF binary; post-route NTT resources are normalized to M16. Circles, squares, and triangles denote LUTs, FFs, and DSPs in (c).

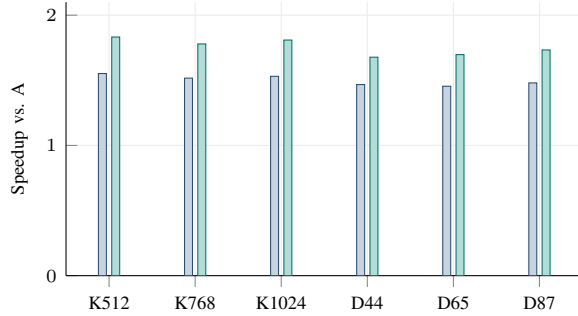


Fig. 7. Six-parameter 200-MHz V80 A/E complete-request speedups. Blue/teal denote one/eight requests; arms use the parameter sweep's matched mapping.

AFU also contains Round and Pointer; it uses 243,073 LUTs, 181,881 FFs, and 106 DSPs and closes at 200 MHz with 0.001 ns setup slack. Board arms select software paths within this all-feature image, so their runtime ratios do not imply per-arm area.

D. Design implications and scope

Keccak shows that fewer instructions can reduce batch throughput; NTT shows that other warps can hide a longer collective. M2 is workload-specific: fewer warps or more transform work could move the knee. Board speedups cover ML-KEM-768/ML-DSA-65 in Fig. 5; the separate sweep covers all six sets. Across that sweep, eight resident warps raise optimized eight-request throughput by $3.798\text{--}4.381\times$ over

TABLE III
INDEPENDENT RV32IM POST-ROUTE CORES AT 200 MHZ. M16 VARIES
KECCAK GRANULARITY; M2 IS MEASURED BOTH ALONE AND WITH
STAGE.

Variant	LUT	FF	DSP	WNS (ns)
M16 base	233,759	136,451	176	+0.078
M16 + Stage	237,073	138,503	176	+0.116
M16 + Round	236,625	142,541	176	+0.052
M16 + Pointer	243,399	139,407	176	+0.080
M2 base	223,964	131,306	106	+0.048
M2 + Stage	226,852	133,368	106	+0.145

one worker. With W8 fixed, 64-request throughput is $0.998\text{--}1.025\times$ its eight-request value. The batches use synchronized waves, not a dynamic queue. Energy is excluded without workload activity.

V. CONCLUSION

Communication-matched warp collectives keep Keccak and NTT state in ordinary registers while fitting a conventional SIMT issue and writeback path. The same serialized NTT unit supports ML-KEM, ML-DSA, and DSA pointwise arithmetic. Complete requests favor Stage over software shuffles, M2 over a peak-throughput M16 bank, and different Keccak boundaries at different concurrency. These results make request-level profiling and matched controls part of the hardware design method, rather than an evaluation afterthought.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “Module-lattice-based digital signature standard,” NIST, Tech. Rep. FIPS 204, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [3] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on $RV\{32,64\}IM\{B\}\{V\}$,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [4] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [5] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [6] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [7] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [8] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [9] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [10] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [11] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [12] R. Yin, Y. Li, and P. Panda, “PacQ: A SIMT microarchitecture for efficient dataflow in hyper-asymmetric GEMMs,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [13] Y. Yao, C. Zhang, C. Qi, R. Chen, J. Wang, Z. Fu, N. Jing, X. Liang, and Z. Song, “SynGPU: Synergizing CUDA and bit-serial tensor cores for vision transformer acceleration on GPU,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [14] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [15] A. Putra, C. M. Duong, A. Q. Pham, and J.-Y. Kim, “ZK-Flex: A flexible and scalable framework for accelerating zero-knowledge proofs,” in *Proc. Design Automation Conf. (DAC)*, 2026.
- [16] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>
- [17] —, “mlds-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlds-native>